



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 6. Работа с датой и временем

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

Модуль datetime

- ⦿ Класс datetime
- ⦿ Класс time
- ⦿ Класс date
- ⦿ Класс timedelta
- ⦿ Формат timestamp

Модуль pytz

Модуль time

Дата и время

- 📶 С датой и временем приходится работать часто:
 - 🕒 Веб-приложения
 - 🗄 Базы данных
 - 📅 Журналирование событий
- 📶 Для этих целей в Python существуют 2 модуля:
 - 🕒 `datetime` – основной модуль для работы с датой и временем
 - 🕒 `time` – предоставляет дополнительные специальные инструменты
- 📶 Модуль `datetime` содержит следующие классы: `datetime`, `date`, `time`, `timedelta`, `tzinfo` – часовые пояса

```
import datetime, time
```

```
print(datetime.datetime.now())
```

```
2021-01-26 13:13:51.544757
```

```
print(time.time())
```

```
1611656031.5447576
```

Объект datetime

Создание объекта datetime:

- ⦿ Для получения текущей даты используется метод `.now()`
- ⦿ Для создания произвольной даты используется конструктор `datetime()` с параметрами: year, month, day, hour, minute, second, microsecond, tzinfo, fold

```
from datetime import datetime
```

```
datetime_now = datetime.now()
```

```
print(datetime_now, type(datetime_now))
```

```
2021-01-27 11:01:34.598662 <class 'datetime.datetime'>
```

```
datetime_random = datetime(1920, 5, 25, 12, 34, 52, 1)
```

```
print(datetime_random, type(datetime_random))
```

```
1920-05-25 12:34:52.000001 <class 'datetime.datetime'>
```

Объекты классов `date` и `time`

- 📡 Получение даты или времени из объекта `datetime`
- 📡 Используются методы `.date()` и `.time()`
 - 🕒 `.date()` – получение даты
 - 🕒 `.time()` – получение времени

```
# datetime_now = 2021-01-27 11:01:34.598662
```

```
date_now = datetime_now.date()  
print(date_now, type(date_now))  
2021-01-27 <class 'datetime.date'>
```

```
time_now = datetime_now.time()  
print(time_now, type(time_now))  
11:19:46.691294 <class 'datetime.time'>
```

Обратите внимание
на типы полученных
значений

Атрибуты классов

- Из объектов `date`, `time` и `datetime` можно извлекать отдельные значения
 - Для этого необходимо обратиться к соответствующему атрибуту

```
# datetime_now = 2021-01-27 11:01:34.598662
```

<code>print(datetime_now.year)</code>	2021
<code>print(datetime_now.month)</code>	1
<code>print(datetime_now.day)</code>	27
<code>print(datetime_now.hour)</code>	11
<code>print(datetime_now.minute)</code>	1
<code>print(datetime_now.second)</code>	34
<code>print(datetime_now.microsecond)</code>	598662

Полный список
атрибутов доступен
в документации

Строковое представление (1 из 2)

- Для работы со строковым представлением объекта `datetime` используются методы `.strptime()` и `.strftime()`
 - `.strptime()` – конвертирует строку в объект `datetime`
 - `.strftime()` – конвертирует объект `datetime`, `time` или `date` в строку
- При конвертировании указывается шаблон

```
date_string = '25.11.2020 23:17:21'
```

```
date_1 = datetime.strptime(date_string, "%d.%m.%Y %H:%M:%S")
```

```
print(date_1, type(date_1))
```

```
2020-11-25 23:17:21 <class 'datetime.datetime'>
```

```
date_string_2 = date_1.strftime("%d-%m")
```

```
print(date_string_2)
```

```
25-11 <class 'str'>
```

Строковое представление (2 из 2)

📡 Шаблон составляется из символов форматирования

📡 Основные символы:

- 🕒 %Y, %m, %d – год, месяц, день
- 🕒 %H, %M, %S – часы, минуты, секунды
- 🕒 %a, %A – сокращенные и полные имена дней недели
- 🕒 %b, %B – сокращенные и полные имена месяца

```
datetime_now = datetime.now()
datetime_now_str = datetime_now.strftime(
    "Date: %a %dth %B %Y, Time: %H:%M"
)
print(datetime_now_str)
Date: Wed 27th January 2021, Time: 13:33
```

Полный список
символов
форматирования
доступен в
документации

Класс `timedelta` (1 из 3)

- 📡 Объект класса `timedelta` представляет собой разницу во времени между двумя объектами `datetime`
- 📡 Для создания объекта `timedelta` используется конструктор с параметрами: `days`, `seconds`, `microseconds`, `milliseconds`, `minutes`, `hours`, `weeks`
 - 🕒 Все параметры по умолчанию равны 0
 - 🕒 Значения параметров `milliseconds`, `minutes`, `hours` и `weeks` конвертируются в `days`, `seconds` и `microseconds`

```
delta = timedelta(days=4, weeks=2, hours=3)

print(delta, type(delta))
18 days, 3:00:00 <class 'datetime.timedelta'>
```

Класс `timedelta` (2 из 3)

- При вычитании одного объекта `datetime` из другого создается объект `timedelta`, представляющий собой разницу во времени между объектами `datetime`
- Для получения прошлой или будущей даты относительно объекта `datetime` используются операции сложения и вычитания

```
datetime_1 = datetime(2020, 5, 15, 18, 0, 0)
datetime_2 = datetime(2020, 5, 17, 15, 0, 0)
```

```
delta = datetime_2 - datetime_1
print(delta)           1 day, 21:00:00
```

```
delta = timedelta(days=4, hours=3)
future = datetime_1 + delta
print(future)          2020-05-19 21:00:00
```

Класс `timedelta` (3 из 3)

 Объекты `timedelta` поддерживают некоторые операции:

- ⦿ Сложение – сумма двух объектов `timedelta`
- ⦿ Вычитание – разница двух объектов `timedelta`
- ⦿ Умножение – произведение объекта `timedelta` на число
- ⦿ Деление – частное двух объектов `timedelta`; результат деления объекта `timedelta` на число
- ⦿ Получение модуля – абсолютное значение объекта `timedelta`

<code>print(delta_1 + delta_2)</code>	15 days, 0:00:00
<code>print(delta_1 - delta_2)</code>	5 days, 0:00:00
<code>print(delta_1 * 2.5)</code>	25 days, 0:00:00
<code>print(delta_1 / delta_2)</code>	2.0
<code>print(delta_1 / 2.2)</code>	4 days, 13:05:27.272727
<code>print(abs(delta_2 - delta_1))</code>	5 days, 0:00:00

Модуль pytz (1 из 3)

- 📡 Объекты **datetime** могут быть двух типов: **naive** и **aware**
 - 🕒 **naive** – объекты, не содержащие информацию о часовых поясах
 - 🕒 **aware** – объекты, содержащие информацию о часовых поясах
- 📡 2 способа работы с часовыми поясами в Python:
 - 🕒 Класс **tzinfo** – встроенный не самый удобный инструмент
 - 🕒 Библиотека **pytz** – популярная библиотека; требует установки
- 📡 Учитывать часовые пояса – хорошая практика

```
import pytz

datetime_1 = datetime(2020, 11, 25, 15, 0, 0)
datetime_2 = datetime(2020, 11, 25, 23, 0, 0)
# Локализация дат
...
print(datetime_1 == datetime_2)    True
```

Значения объектов
равны, так как они
задают одну и ту же
дату в разных
часовых поясах

Модуль pytz (2 из 3)

 pytz предлагает 2 основных метода:

- ⦿ .localize() – служит для задания принадлежности часовому поясу
- ⦿ .astimezone() – позволяет переходить между часовыми поясами

```
dt_1 = datetime(2020, 11, 25, 15, 0, 0)
print(dt_1)                2020-11-25 15:00:00
```

```
us_ny_tz = pytz.timezone('America/New_York')
dt_1 = us_ny_tz.localize(dt_1)
print(dt_1)                2020-11-25 15:00:00-05:00
```

```
eu_msc_tz = pytz.timezone('Europe/Moscow')
dt_2 = dt_1.astimezone(us_ny_tz)
print(dt_2)                2020-11-25 23:00:00+03:00
```

Модуль pytz (3 из 3)

- 📶 Часовой пояс – смещение относительно нулевого часового пояса
- 📶 Нулевой часовой пояс – UTC (+00:00)
- 📶 Для уменьшения вероятности ошибок все операции стоит проводить в UTC

```
# datetime_1 = 2020-11-25 15:00:00-05:00
# datetime_2 = 2020-11-25 23:00:00+03:00

dt_1_utc = datetime_1.astimezone(pytz.utc)
dt_2_utc = datetime_2.astimezone(pytz.utc)
print(dt_1_utc)          2020-11-25 20:00:00+00:00
print(dt_2_utc)          2020-11-25 20:00:00+00:00
```

Объект datetime в
часовом поясе



Объект datetime в
нулевом часовом
поясе (UTC)



Операции



Перевод в
нужный часовой
пояс

Формат timestamp

- 📡 В Python существует возможность работать с датой и временем в представлении UNIX
 - 🕒 Представляет собой количество секунд, прошедших с 01.01.1970 UTC
- 📡 Данный формат (**timestamp**) используется при хранении данных и в простейших операциях
- 📡 В классе **datetime** существуют специальные методы
 - 🕒 `.timestamp()` – принимает **datetime** и возвращает **timestamp**
 - 🕒 `.fromtimestamp()` – принимает **timestamp** и возвращает **datetime**

```
datetime_1 = datetime(2020, 11, 25, 15, 0, 0)
print(datetime_1)                2020-11-25 15:00:00
timestamp_1 = datetime.timestamp(datetime_1)
print(timestamp_1)               1606305600.0
datetime_2 = datetime.fromtimestamp(timestamp_1 - 3600)
print(datetime_2)               2020-11-25 14:00:00
```

Модуль time

- 📡 Большинство задач при работе с датой и временем можно выполнить при помощи модуля **datetime**
- 📡 Модуль **time** предоставляет инструменты для работы с процессами, потоками и временем выполнения программ
 - ⦿ `.sleep()` – останавливает выполнение программы на указанное время
 - ⦿ `.process_time()` – возвращает время выполнения текущего процесса

```
import time

start = time.process_time()

for i in range(100000):
    print(i, end=' ')
print()

end = time.process_time()
print(end - start)          0.671875
```

По умолчанию
используйте модуль
datetime

Пример программы

Пример программы на сервере

```
datetime_now = datetime.now()

eu_msc_tz = pytz.timezone('Europe/Moscow')
datetime_now = eu_msc_tz.localize(datetime_now)
datetime_now = datetime_now.astimezone(pytz.utc)
datetime_now = datetime_now.timestamp()

# Получили из БД дату начала курса
course_date_ts = 1612969200

if course_date_ts > datetime_now:
    print('Информация о курсе отображается')
else:
    print('Информация о курсе не отображается')
```

Получение даты и
локализация



Перевод в нулевой
часовой пояс и
формат timestamp



Сравнение с датой
начала курса



Вывод
соответствующего
сообщения



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 6. Обработка исключений

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

- 📡 Понятие «исключение»
- 📡 Сообщение об ошибке
- 📡 Иерархия исключений
- 📡 Конструкция try – except
 - 🕒 Блок try
 - 🕒 Блок except
 - 🕒 Блок else
 - 🕒 Блок finally
- 📡 Ключевое слово raise

Исключения

- 📡 Исключение – событие времени выполнения, которое может привести к ошибке
- 📡 Необработанное исключение вызывает ошибку
 - 🕒 Поэтому важно обрабатывать исключения
- 📡 Ошибки бывают разных типов

```
print(a)
```

```
D:\Работа\PYTHON\test\venv\Scripts\python.exe D:/Работа/PYTHON/test/test.py
Traceback (most recent call last):
  File "D:/Работа/PYTHON/test/test.py", line 242, in <module>
    print(a)
NameError: name 'a' is not defined

Process finished with exit code 1
```

Сообщение об ошибке

Пример описания ошибки

- ⦿ Выполненная команда с абсолютными путями

```
D:\Работа\PYTHON\test\venv\Scripts\python.exe D:/Работа/PYTHON/test/test.py
Traceback (most recent call last):
  File "D:/Работа/PYTHON/test/test.py", line 242, in <module>
    print(a)
NameError: name 'a' is not defined

Process finished with exit code 1
```



Выполненная команда

Сообщение об ошибке

Пример описания ошибки

- Имя файла, в котором возникла ошибка

```
D:\Работа\PYTHON\test\venv\Scripts\python.exe D:/Работа/PYTHON/test/test.py
Traceback (most recent call last):
  File "D:/Работа/PYTHON/test/test.py", line 242, in <module>
    print(a)
NameError: name 'a' is not defined

Process finished with exit code 1
```



Имя файла

Сообщение об ошибке

Пример описания ошибки

- Номер строки, в которой возникла ошибка

```
D:\Работа\PYTHON\test\venv\Scripts\python.exe D:/Работа/PYTHON/test/test.py
Traceback (most recent call last):
  File "D:/Работа/PYTHON/test/test.py", line 242, in <module>
    print(a)
NameError: name 'a' is not defined

Process finished with exit code 1
```



Номер строки

Сообщение об ошибке

Пример описания ошибки

- ⦿ Тип возникшей ошибки

```
D:\Работа\PYTHON\test\venv\Scripts\python.exe D:/Работа/PYTHON/test/test.py
Traceback (most recent call last):
  File "D:/Работа/PYTHON/test/test.py", line 242, in <module>
    print(a)
NameError: name 'a' is not defined
Process finished with exit code 1
```



Тип ошибки

Сообщение об ошибке

Пример описания ошибки

- ⦿ Подробное описание ошибки

```
D:\Работа\PYTHON\test\venv\Scripts\python.exe D:/Работа/PYTHON/test/test.py
Traceback (most recent call last):
  File "D:/Работа/PYTHON/test/test.py", line 242, in <module>
    print(a)
NameError: name 'a' is not defined
Process finished with exit code 1
```



Описание ошибки

Сообщение об ошибке

Пример описания ошибки

- Код, с которым завершился процесс

```
D:\Работа\PYTHON\test\venv\Scripts\python.exe D:/Работа/PYTHON/test/test.py
Traceback (most recent call last):
  File "D:/Работа/PYTHON/test/test.py", line 242, in <module>
    print(a)
NameError: name 'a' is not defined

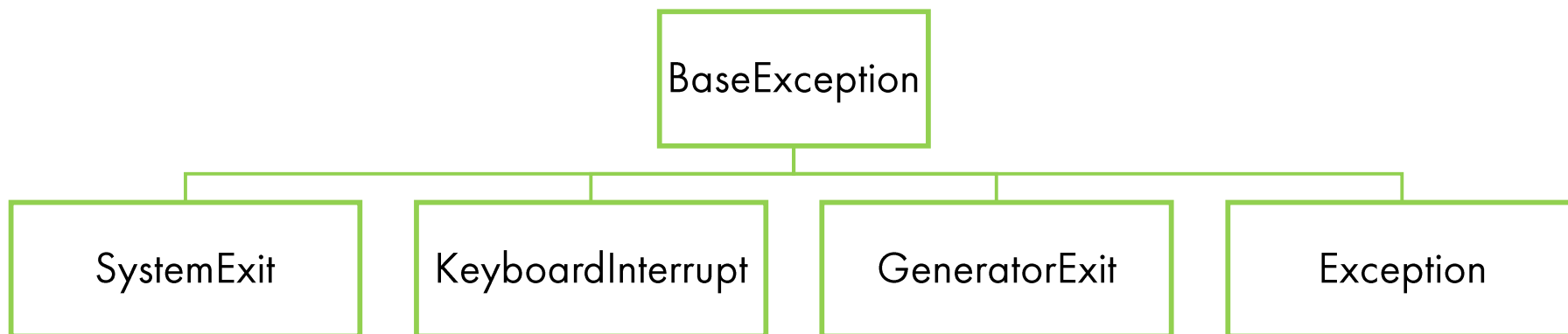
Process finished with exit code 1
```

Код завершения процесса

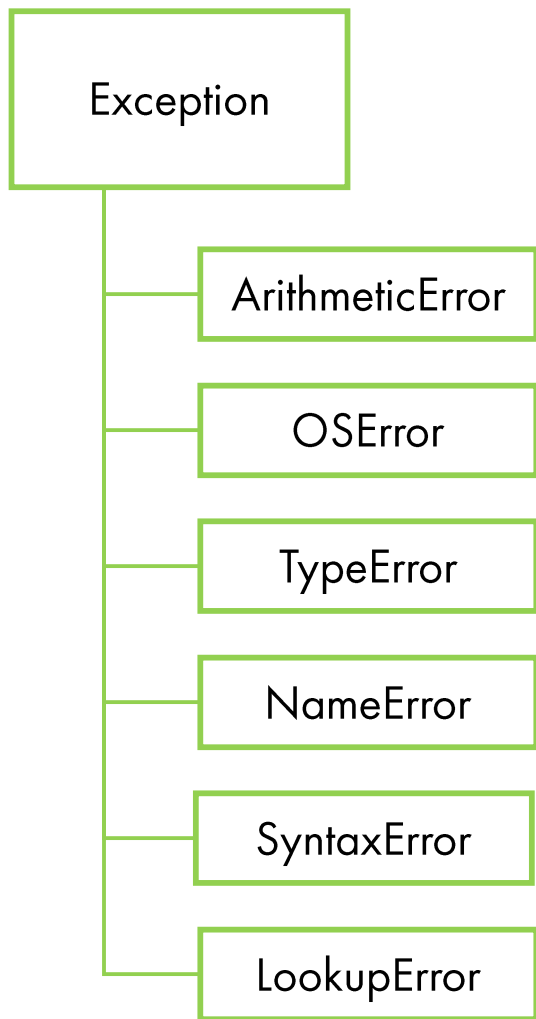
Иерархия исключений (1 из 2)

📡 BaseException – родительский для всех встроенных исключений класс

- ⦿ SystemExit – вызывается функцией sys.exit()
- ⦿ KeyboardInterrupt – возникает при прерывании выполнения программы пользователем (CTRL + C)
- ⦿ GeneratorExit – возникает при закрытии генераторов



Иерархия исключений (2 из 2)



Exception – родительский класс для обычных (не системных) исключений

- ArithmeticError – арифметические ошибки
- OSError – исключения, возникающие при работе с операционной системой
- TypeError – несоответствие типов
- NameError – отсутствие переменной с указанным именем
- SyntaxError – синтаксические ошибки
- LookupError – неправильный индекс или ключ

Полная иерархия исключений гораздо больше

Конструкция `try – catch`

- Для обработки исключений используется конструкция `try – except`
- Любой код, который может привести к ошибке, стоит помещать в конструкцию `try – except`
- В блок `try` помещается код, в котором могут возникать ошибки
- В блоке `except` содержится код, который выполняется при возникновении указанного исключения

```
summ = 300
count = 0

try:
    mean = summ / count
    print('Среднее значение:', mean)
except ZeroDivisionError:
    print("Деление на ноль!")
```

Данная программа
завершится корректно!

Тип предполагаемой
ошибки

Блок **except** (1 из 3)

 Блоков **except** может быть несколько

- ⦿ Обработывается несколько потенциальных исключений разных типов
- ⦿ Каждое исключение обрабатывается по-разному

```
a = int(input())
b = int(input())

try:
    result_div = a / b
    result_log = math.log(a, b)
except ZeroDivisionError:
    print('Деление на ноль!')
except ValueError:
    print('Ошибка в логарифме!')
```

Исключения
ZeroDivisionError и
ValueError обрабатываются
раздельно

Блок `except` (2 из 3)

- 📡 Один блок `except` может обрабатывать разные исключения
 - ⦿ Обрабатывается несколько потенциальных исключений разных типов
 - ⦿ Все исключения обрабатываются одинаково

```
a = int(input())
b = int(input())

try:
    result_div = a / b
    result_log = math.log(a, b)
except (ZeroDivisionError, ValueError):
    print('Ошибка при вычислении')
```

Исключения
ZeroDivisionError и
ValueError
обрабатываются вместе

Блок `except` (3 из 3)

- Если не указывать тип исключения, то данный блок `except` будет обрабатывать все исключения
 - Такой блок `except` может быть только последним
 - Избегайте подобных блоков

```
a = int(input())
b = int(input())

try:
    result_div = a / b
    result_log = math.log(a, b)
except (ZeroDivisionError, ValueError):
    print('Ошибка при вычислении')
except:
    print('Неопознанная ошибка!')
```


Блок `else`

- 📡 Код, содержащийся в блоке `else`, выполняется в том случае, если исключений не было
- 📡 Блок `else` идёт после всех блоков `except`

```
a = int(input())
b = int(input())

try:
    result_div = a / b
    result_log = math.log(a, b)
except ZeroDivisionError:
    print('Деление на ноль!')
except ValueError:
    print('Ошибка в логарифме!')
else:
    print('Ошибок нет')
```

Блок **finally**

- 🌀 Код, содержащийся в блоке **finally**, выполняется в любом случае
- 🌀 Блок **finally** идёт после всех остальных блоков

```
a = int(input())
b = int(input())

try:
    result_div = a / b
    result_log = math.log(a, b)
except ZeroDivisionError:
    print('Деление на ноль!')
except ValueError:
    print('Ошибка в логарифме!')
else:
    print('Ошибок нет')
finally:
    print('Расчёты окончены')
```

Ключевое слово `raise` (1 из 2)

📡 Ключевое слово `raise` позволяет вручную вызывать исключения

📡 После `raise` можно указать:

- 🕒 Тип вызываемого исключения
- 🕒 Тип вызываемого исключения и описание ошибки

```
try:  
    raise TypeError("Несоответствие типов!")  
except TypeError:  
    print("Ошибка!")
```

📡 Пример необработанного исключения, вызванного вручную:

```
File "D:/Работа/PYTHON/test/test.py", line 269, in <module>  
    raise TypeError("Несоответствие типов!")  
TypeError: Несоответствие типов!
```

Ключевое слово `raise` (2 из 2)

- 📡 В Python существует возможность создать собственный класс исключений
 - 🕒 Далее данное исключение вызывается при помощи `raise`
- 📡 Созданный класс исключений должен быть прямым или косвенным потомком класса `Exception`

```
class Custom_exception(Exception):  
    def __init__(self, message):  
        self.message = message
```

Создание собственного
класса исключений

```
try:
```

```
    x = int(input())
```

```
    if x == 10:
```

```
        raise Custom_exception("Делить на 10 нельзя!")
```

```
    result = 100 / x
```

```
except Custom_exception as exception:
```

```
    print(exception)
```

Вызов исключения

Обработка исключения

Цепочки исключений

- Если код внутри блоков **except**, **else** и **finally** может привести к ошибке, то этот код также нужно поместить в конструкцию **try - except**

```
a = int(input())
b = int(input())

try:
    result_div = a / b
except ZeroDivisionError:
    try:
        print(1/0)
    except ZeroDivisionError:
        print('Деление на ноль!')
    print('Деление на ноль!')
```

Если этого не сделать, то возникнет сообщение об ошибке, которая возникла при обработке другого исключения

Документация

 Информацию о типах исключений, которые могут возникнуть в каком-либо коде можно посмотреть:

- ⦿ В иерархии исключений
- ⦿ В документации

```
def open(file, mode='r', buffering=None, encoding=None, errors=None, newline=None, closefd=True):  
    """  
    Open file and return a stream. Raise OSError upon failure.
```

```
try:  
    with open("dir1/text1.json", 'r') as file:  
        json.dump(user_data, file, indent=4)  
except OSError as err:  
    print(err)
```

Итоги

В данной лекции были рассмотрены следующие темы:

-  Понятие «исключение»
-  Сообщение об ошибке
-  Иерархия исключений
-  Конструкция try – except
 -  Блок try
 -  Блок except
 -  Блок else
 -  Блок finally
-  Ключевое слово raise